

上海大学

计算机工程与科学学院

2024 学年《系统结构》研讨报告

课题名称： ARM 平台上密码学算子性能优化

课题成员：

姓 名	文档撰写工作 分工	研究开发工作 分工	对课题的贡献 百分比
聂冬亮	MD5 算子报告	MD5 算子优化	25%
赵文宇	报告润色	PPT 制作与汇报	25%
邹佳乐	SHA256 算子报告	SHA256 算子优化	25%
韦杰恒	AES 算子报告	AES 算子优化	25%

评语：

得分：

基准程序

程序优化方法

- 循环完全展开
- 函数内联与减少中间临时变量
- 预处理期宏展开
- 使用尽量小的数据类型
- 使用高效库函数替代多次低效调用
- 实验结果对比（程序优化部分）

ARM平台上的程序性能优化方法

- 数据对齐优化
- 软件预取优化
- 预计算查表优化
- 使用高效库函数替代多次低效调用（ARM平台）
- 实验结果对比和总结（ARM平台优化部分）

基准程序

基础版本实现了三种经典密码学算子的完整计算框架，作为后续优化对比的参照基准。

AES实现了AES-128/192/256在ECB模式下的加解密操作。核心操作包含SubBytes（S盒替换）、ShiftRows（行移位）、MixColumns（列混淆）和AddRoundKey（轮密钥加），每个操作在 4×4 的状态矩阵上进行。编译选项为-O0，旨在暴露最真实的硬件执行瓶颈。1MB数据Baseline耗时298.299ms。

SHA-256实现了SHA-2家族的核心哈希算法，将任意长度的输入压缩为256位摘要。核心是64轮迭代的压缩函数，包含循环右移、异或、与、非等位运算和模 2^{32} 加法。编译选项为-O0。1MB数据Baseline耗时100.279ms。

MD5实现了广泛使用的消息摘要算法，将输入压缩为128位摘要。核心是4阶段 $\times$ 16轮的主循环，每轮使用不同的非线性函数（F/G/H/I）和消息字索引。编译选项为-O3。1MB数据Baseline耗时6,523,497ns。

程序优化方法

循环完全展开

循环展开是一种通过减少循环控制开销来提升程序性能的优化技术。在循环结构中，每次迭代都需要执行循环变量自增（`i++`）、条件判断（`i < N`）和跳转指令。当循环体本身较小而迭代次数较多时，循环控制指令占总指令数的比例较高，降低循环控制频率可以有效提升执行效率。

本实验的循环完全展开体现在以下方面：

AES的ShiftRows展开：原函数使用三层嵌套循环实现行移位操作，包含循环变量`i/k/s`的维护和中间数组`tmp[4]`。优化后直接以硬编码赋值代替循环，消除了所有循环控制开销。

AES的SubBytes展开：原函数使用双层循环遍历16个字节进行S盒替换。优化后展开为16路独立的查表赋值语句。在`-O0`下编译器不会自动做此展开，手动展开使得CPU可以并行执行16次独立的内存读操作。

AES的AddRoundKey展开：原函数在每一轮中通过 $4 \times Nb \times r$ 乘法计算轮密钥位置，循环内重复计算索引乘法。优化后预计算轮密钥起始指针，将16次XOR操作完全展开，消除乘法运算的同时使16次XOR可以无依赖并行执行。

MD5的主循环完全展开：原函数使用统一64次循环迭代，每轮需通过if-else分支判断当前阶段是F/G/H/I，并计算%16取模获取消息字索引。优化后将64轮完全展开为固定MD5_STEP宏序列，彻底消除了循环控制、分支跳转和取模运算。

代码对比（以AES ShiftRows为例）：

优化前：

```
1 void inv_shift_rows(uint8_t *state) {
2     uint8_t i, k, s;
3     uint8_t tmp[4];
4     for (i = 1; i < 4; i++) {
5         for(s = 0; s < i ; s++) tmp[s] = state[Nb * i + Nb - i + s];
6         for(k = Nb-1; k > i - 1; k--) state[Nb*i+k] = state[Nb*i+k-
7         i];
8         for(s = 0; s < i ; s++) state[Nb * i + s] = tmp[s];
9     }
```

优化后：

```
1 void inv_shift_rows(uint8_t *state) {  
2     uint8_t tmp;  
3     tmp = state[7]; state[7] = state[6]; state[6] = state[5];  
4     state[5] = state[4]; state[4] = tmp;  
5     tmp = state[8]; state[8] = state[10]; state[10] = tmp;  
6     tmp = state[9]; state[9] = state[11]; state[11] = tmp;  
7     tmp = state[12]; state[12] = state[13]; state[13] = state[14];  
8     state[14] = state[15]; state[15] = tmp;  
9 }
```

AES各优化阶段运行时间 (-O0, 单位ms) :

数据规模	Baseline	ShiftRows 展开	SubBytes 展开	AddRoundKey 展开	MixColumns 内联
128 B	0.056	0.045	0.043	0.039	0.034
256 B	0.090	0.114	0.072	0.058	0.048
512 B	0.180	0.138	0.129	0.112	0.092
1 KB	0.368	0.331	0.259	0.225	0.182
2 KB	0.624	0.550	0.520	0.456	0.357
4 KB	1.165	1.076	1.020	0.880	0.685
8 KB	2.353	2.102	2.033	1.710	1.442
16 KB	4.638	4.197	3.988	3.437	2.933
32 KB	9.320	8.480	7.725	6.990	5.537
64 KB	18.208	16.740	15.457	13.427	10.999
128 KB	36.904	34.169	31.491	27.475	22.744
256 KB	73.600	66.834	62.447	55.598	46.606
512 KB	150.030	135.053	125.339	111.945	107.359
1 MB	298.299	269.063	249.496	222.601	182.428

AES各优化版本相对于基线的加速比：

数据规模	ShiftRows展开	SubBytes展开	AddRoundKey展开	MixColumns内联
128 B	1.244	1.302	1.436	1.647
256 B	0.789	1.250	1.552	1.875
512 B	1.304	1.395	1.607	1.957
1 KB	1.112	1.421	1.636	2.022
2 KB	1.135	1.200	1.368	1.748
4 KB	1.083	1.142	1.324	1.701
8 KB	1.119	1.157	1.376	1.632
16 KB	1.105	1.163	1.349	1.581
32 KB	1.099	1.206	1.333	1.683
64 KB	1.088	1.178	1.356	1.655
128 KB	1.080	1.172	1.343	1.623
256 KB	1.101	1.179	1.324	1.579
512 KB	1.111	1.197	1.340	1.397
1 MB	1.109	1.196	1.340	1.635

函数内联与减少中间临时变量

函数内联是将被调用函数的代码直接嵌入到调用处，从而消除函数调用开销（参数压栈、BL跳转、栈恢复等指令）的优化手段。减少中间临时变量则通过直接写回目标位置来消除冗余的内存读写操作。

本实验的函数内联体现在AES的InvMixColumns优化中。原函数通过coef_mult辅助函数和数组参数完成列混淆，每次调用需建立数组col[4]和res[4]存储中间结果，函数返回后再写回state矩阵。优化后将coef_mult直接内联到主函数中，用编译期常量0x0e/0x09/0x0d/0x0b替代数组a[]的查表，将col[4]和res[4]替换为局部变量s0/s1/s2/s3直接写回state。

代码对比：

优化前：

```

1 void inv_mix_columns(uint8_t *state) {
2     uint8_t a[] = {0x0e, 0x09, 0x0d, 0x0b};
3     uint8_t col[4], res[4];
4     for (j = 0; j < Nb; j++) {
5         for (i = 0; i < 4; i++) col[i] = state[Nb*i+j];
6         coef_mult(a, col, res);
7         for (i = 0; i < 4; i++) state[Nb*i+j] = res[i];
8     }
9 }

```

优化后：

```

1 void inv_mix_columns(uint8_t *state) {
2     uint8_t s0, s1, s2, s3;
3     for (int j = 0; j < 4; j++) {
4         s0 = state[j]; s1 = state[j+4]; s2 = state[j+8]; s3 =
state[j+12];
5         state[j] =
gmult(0x0e,s0)^gmult(0x0b,s1)^gmult(0x0d,s2)^gmult(0x09,s3);
6         state[j+4] =
gmult(0x09,s0)^gmult(0x0e,s1)^gmult(0x0b,s2)^gmult(0x0d,s3);
7         state[j+8] =
gmult(0x0d,s0)^gmult(0x09,s1)^gmult(0x0e,s2)^gmult(0x0b,s3);
8         state[j+12]=
gmult(0x0b,s0)^gmult(0x0d,s1)^gmult(0x09,s2)^gmult(0x0e,s3);
9     }
10 }

```

这一优化在AES的1MB数据上贡献了1.22×的单步加速（222.601ms→182.428ms），是AES优化中单步收益最大的阶段。

预处理期宏展开

预处理期宏展开是利用C/C++预处理器的文本替换机制，在编译器工作前就将代码直接“粘贴”到目标位置，从而阻止编译器产生函数调用指令的优化手段。这在低优化级别（尤其是-O0）下效果显著，因为-O0下inline关键字会被编译器忽略。

本实验的宏展开体现在SHA-256优化中。在-O0编译环境下，原始代码中声明的static inline位运算函数被编译器强制生成为真实函数调用。SHA-256的64轮压缩函数中，每轮需调用r、Ch、Ma、S0、_S1等多个位运算函数，1MB数据包含约15,625个512位数据块，累计产生数百万次函数栈帧的压栈/出栈操作。

优化后将所有位运算函数改写为宏定义（#define），在主循环中配合4×手动步长展开。

代码对比：

优化前 (inline函数, -O0下生成函数调用) :

```
1 static inline uint32_t _r(uint32_t x, uint8_t n) { return ((x >> n) |  
_shw(x, 32 - n)); }  
2 static inline uint32_t _Ch(uint32_t x, uint32_t y, uint32_t z) {  
return ((x & y) ^ ((~x) & z)); }  
3 static inline uint32_t _Ma(uint32_t x, uint32_t y, uint32_t z) {  
return ((x & y) ^ (x & z) ^ (y & z)); }
```

优化后 (宏定义, 预处理期完成替换) :

```
1 #define _r(x, n) (((x) >> (n)) | _shw(x, 32 - (n)))  
2 #define _Ch(x, y, z) (((x) & (y)) ^ ((~(x)) & (z)))  
3 #define _Ma(x, y, z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))  
4 #define _S0(x) (_r((x), 2) ^ _r((x), 13) ^ _r((x), 22))  
5 #define _S1(x) (_r((x), 6) ^ _r((x), 11) ^ _r((x), 25))
```

1MB数据从100.279ms暴降至29.104ms, 加速约3.45×, 是全部实验中单步加速比最大的优化。

使用尽量小的数据类型

能够使用字符型定义的变量, 就不使用整型变量; 能够使用整型变量定义的变量就不用长整型。但在本实验中, AES、SHA-256和MD5均使用了标准规定的精确位宽类型 (uint8_t、uint32_t等), 无法进一步压缩数据类型。SHA-256在uint32_t上消除了冗余的& 0xff和& 0xffffffff掩码操作, 因为在32位无符号整数类型上, 这些掩码是语义层面冗余的。

使用高效库函数替代多次低效调用

本实验主要在SHA-256的Padding优化中体现了这一方法。原代码使用for循环逐字节将缓冲区清零, 优化后改为标准库memset()函数。编译器可对memset生成更高效的SIMD宽加载/存储指令。

```
1 // 优化前  
2 for (i = j + 1; i < sizeof(ctx->buf); i++) { ctx->buf[i] = 0x00; }  
3  
4 // 优化后  
5 memset(&ctx->buf[j + 1], 0x00, sizeof(ctx->buf) - (j + 1));
```

不过由于SHA-256的Padding阶段最多只需补齐1~64字节, 在-O0下memset自身的函数调用开销与for循环开销相当, 该优化在SHA-256上收益有限 (1MB从100.279ms变为99.696ms)。

实验结果对比（程序优化部分）

AES各优化阶段（-O0，1MB数据）：

优化版本	耗时(ms)	相对Baseline加速比	相对上一阶段加速比
Baseline	298.299	—	—
+ShiftRows展开	269.063	1.11×	1.11×
+SubBytes展开	249.496	1.20×	1.08×
+AddRoundKey展开	222.601	1.34×	1.12×
+MixColumns内联	182.428	1.64×	1.22×

SHA-256各优化阶段（-O0，1MB数据）：

优化版本	耗时(ms)	加速比
Baseline	100.279	—
+冗余掩码消除+分支预测	100.009	1.00×
+Padding memset	99.696	1.01×
+数据预取	102.344	0.98×
+宏替换+循环展开	29.104	3.45×
全部优化	30.515	3.29×

MD5各优化阶段（-O3，1MB数据）：

优化版本	耗时(ns)	加速比
Baseline	6,523,497	—
+Padding O(1)	6,354,875	1.03×
+对齐分配+复用	6,369,572	1.02×
+数据预取+对齐提示	6,590,318	0.99×
+主循环完全展开	4,942,816	1.32×
全部优化	4,855,467	1.34×

ARM平台上的程序性能优化方法

数据对齐优化

数据对齐是指编译器在分配内存空间时按照一定的规则将数据存储在特定位置，以符合特定对齐要求的做法。对齐的数据可以让CPU在单次访存操作中完整读取所需数据，而非对齐数据可能跨越两个缓存行（Cache Line），一次访问需要两次内存操作，产生额外的延迟。

本实验在MD5优化中引入了64字节对齐分配。MD5以64字节为单位处理数据块，当数据块起始地址对齐到64字节（ARM Cortex-A72的Cache Line大小）时，一个完整的64字节数据块正好落在一个Cache Line中，消除了跨行访问的开销。同时引入缓冲区复用机制，记录msg_size，仅在需要更大空间时重新分配，避免重复的malloc/free开销。

```
1 // 优化前
2 msg = (uint8_t*)malloc(new_len + 8);
3
4 // 优化后
5 if (msg_size < new_len + 8) {
6     free(msg);
7     if (posix_memalign((void **)&msg, 64, new_len + 8) != 0)
8         msg = (uint8_t*)malloc(new_len + 8);
9     msg_size = new_len + 8;
10 }
```

本优化在MD5上带来约1.02×的加速（6,523,497ns→6,369,572ns）。

软件预取优化

预取技术可以预先将数据加载到缓存中，从而减少缓存未命中（Cache Miss）的次数。软件预取使用特定指令（如GCC的__builtin_prefetch）指示硬件提前将后续需要用到数据加载到缓存。预取距离需要恰当选择：距离过大会导致预取的数据在使用前被挤出缓存，距离过小则数据尚未到达Cache，CPU仍需等待。

本实验在MD5和SHA-256中均尝试了软件预取优化。

MD5的预取：在遍历512位数据块时，使用 `__builtin_prefetch(msg + offset + 64, 0, 3)` 提前将下一个64字节块加载到缓存。同时使用 `__builtin_assume_aligned` 告知编译器数据已16字节对齐。

```

1  for (offset = 0; offset < new_len; offset += 64) {
2      if (offset + 64 < new_len)
3          __builtin_prefetch(msg + offset + 64, 0, 3);
4      w = (uint32_t *)__builtin_assume_aligned(msg + offset, 16);
5  }

```

SHA-256的预取：在遍历输入字节时预取后续64字节。

```

1  if (i + 64 < len) {
2      __builtin_prefetch(&bytes[i + 64], 0, 3);
3  }

```

MD5在-O3下编译器已经生成了较好的预取指令，手动预取在x86_64上收益不明显（1MB：6,523,497ns→6,590,318ns，略微变慢）。SHA-256在-O0下因代码冗余，预取指令反而污染了缓存线（1MB：100.279ms→102.344ms，变慢2%）。

在ARM Cortex-A72上，因内存延迟更大，预计预取优化会取得更好的效果。

预计算查表优化

预计算查表优化的基本原理是将与输入数据无关、仅依赖于参数的计算结果提前求出并存储，后续执行时直接通过数组索引获取，避免重复计算。这是空间换时间思想的典型应用。

本实验主要体现在AES的S盒查表和AddRoundKey的指针预计算上。AES的SubBytes通过inv_s_box表进行16字节替换，这本身就是查表操作。AddRoundKey中预计算round_key = w + (16 * r)避免了循环内反复计算4×Nb×r乘法。此外，MD5的64轮完全展开本质上是将k[i]常量数组索引和r[i]移位量直接编译到指令中，也是一种预计算思想的体现。

使用高效库函数替代多次低效调用（ARM平台）

本实验在SHA-256的Padding阶段使用了memset替代手写for循环。但在ARM平台上，当处理的数据量极小（1~64字节）且编译优化级别为-O0时，memset的函数调用开销抵消了优势。数据在更大规模的处理中（如MD5的批量数据拷贝），memset/memcpy的优势才能体现。

实验结果对比和总结（ARM平台优化部分）

各算子最终优化效果（1MB数据）：

算子	Baseline	优化后	加速比	主要贡献优化
AES (-O0)	298.299 ms	182.428 ms	1.64×	MixColumns内联+常量替换
SHA-256 (-O0)	100.279 ms	30.515 ms	3.29×	宏替换+循环步长展开
MD5 (-O3)	6,523,497 ns	4,855,467 ns	1.34×	64轮主循环完全展开

优化方法汇总：

优化手段	适用算子	收益评估	适用条件
循环完全展开	AES / MD5 / SHA-256	高	固定迭代次数的小循环 (≤64轮)
函数内联+消临时数组	AES (MixColumns)	高	频繁调用的小函数、有中间数组
预处理期宏替换	SHA-256	高	-O0下inline失效、位运算密集
预计算指针消乘法	AES (AddRoundKey)	中	循环内有重复乘法索引计算
数据预取	SHA-256 / MD5	中(ARM)~低(x86)	数据规模超过缓存容量
内存对齐分配	MD5	中	按Cache Line处理的数据
分支预测提示	SHA-256	低	极偏斜的分支
冗余掩码消除	SHA-256	低 (免费优化)	类型安全的整数位运算
求余→位运算	MD5 (Padding)	低	除以2的幂取余的场景

通过上述比较可以得出，SHA-256的宏替换+循环展开优化效果最为显著，在-O0下取得了3.45×的单步加速，说明当系统存在结构性瓶颈（数百万次函数调用）时，从根源上消除瓶颈比微观优化重要得多。AES的MixColumns内联贡献了1.22×的单步加速，函数调用和临时数组在-O0下的开销不容忽视。MD5的循环完全展开在-O3下仍有1.32×的加速，说明手动消除分支和取模是编译器无法替代的优化。